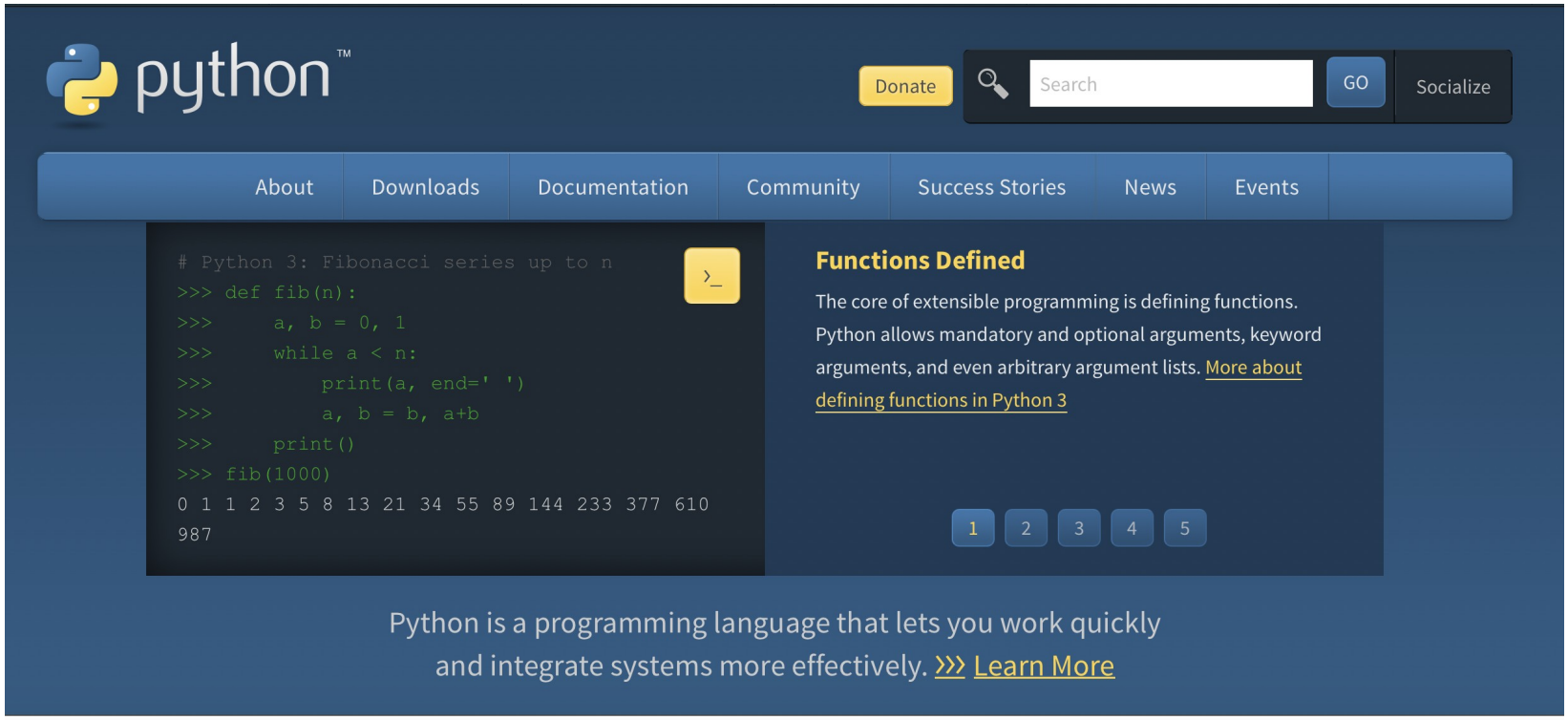
DATA 211– Introduction to Data Science

LECTURE 2: INTRODUCTION TO PYTHON: BASICS

What is Python?

- Python is a high-level, interpreted, and general-purpose programming language known for:
  - Simple and readable syntax
  - Large standard library
  - Strong community support
  - Interpreted Language
  - General-purpose
  - Extensive applications in data science, web development, automation, and more.

Python Installation



https://www.python.org

Python Installation

1. On Windows

1. **Open Command Prompt**
   - Press `Win + R` → type `cmd` → press **Enter**.
2. **Check Python version**

   Type:

```
c

python --version
```

   or

```
cmd

python -V
```

Python Installation

2. On macOS or Linux

1. **Open Terminal**
   - On macOS: `Command + Space` → type `Terminal`
   - On Linux: Open the **Terminal** app
2. **Check Python version**

```bash
python3 --version
```

(macOS/Linux often use `python3` instead of `python`.)

3. **Verify `pip`**

```bash
pip3 --version
```

Running Python

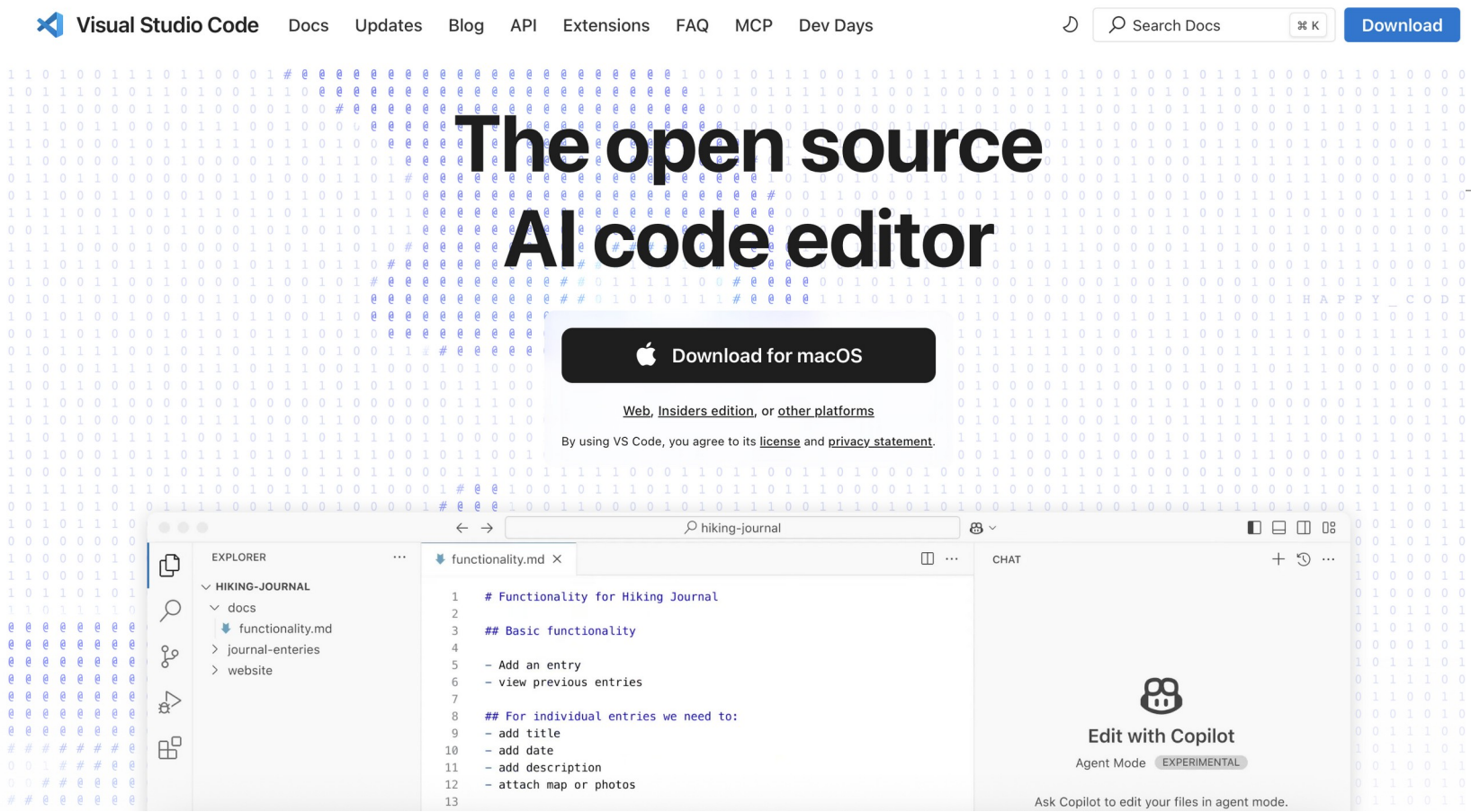
```
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
Usmans-MacBook-Pro:~ uq$ python3
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-
1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Print function

- In Python, the print() function is used to send output to the standard output stream. You can print strings, numbers, expressions, or even variables.

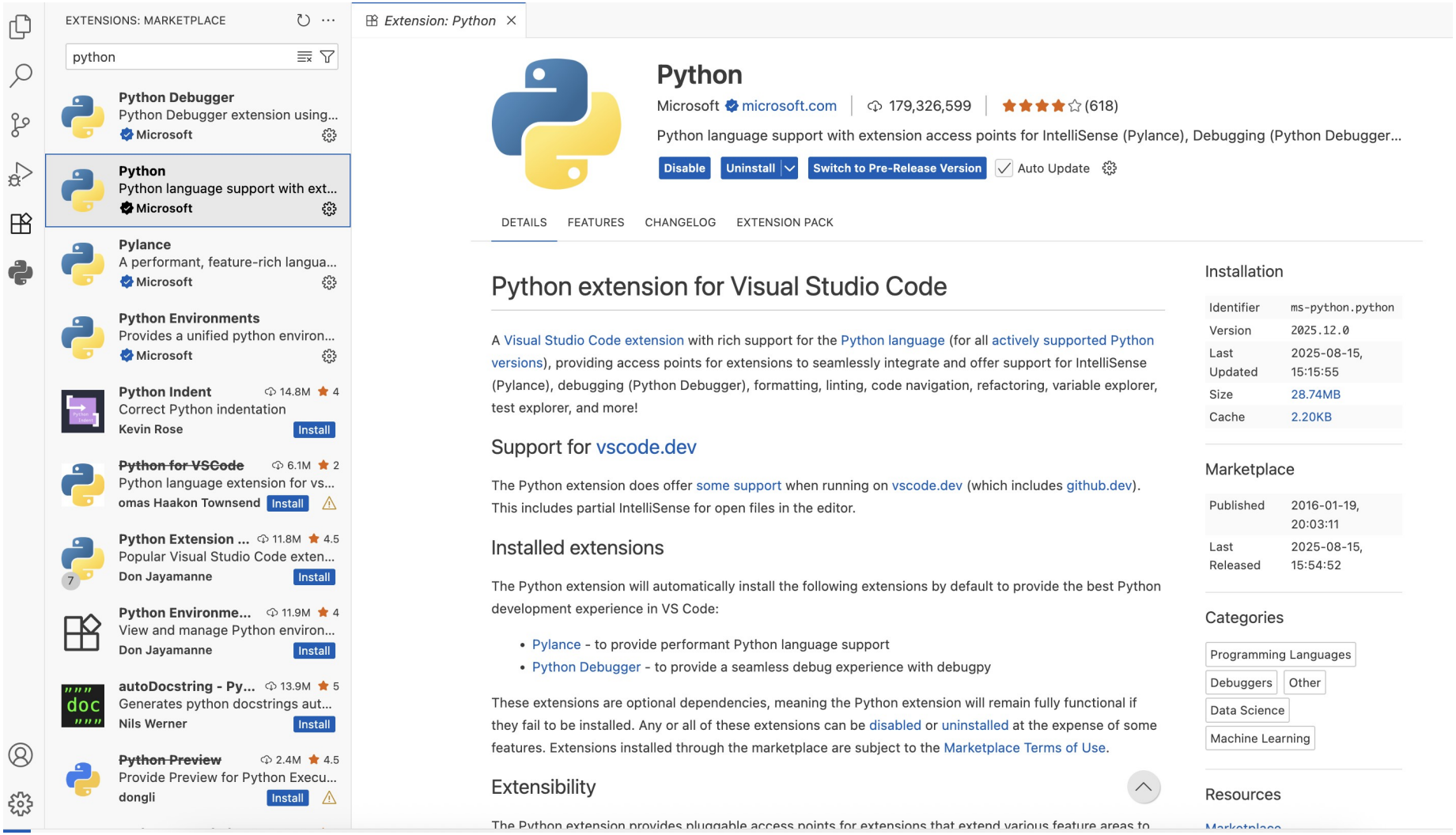
```
# This code prints a welcome message to the console.
# It confirms that the code execution environment is working properly.
# Useful as a first step to check setup before running other logic.
print('Hello, World')
```

```
Hello, World
```



Download from https://code.visualstudio.com

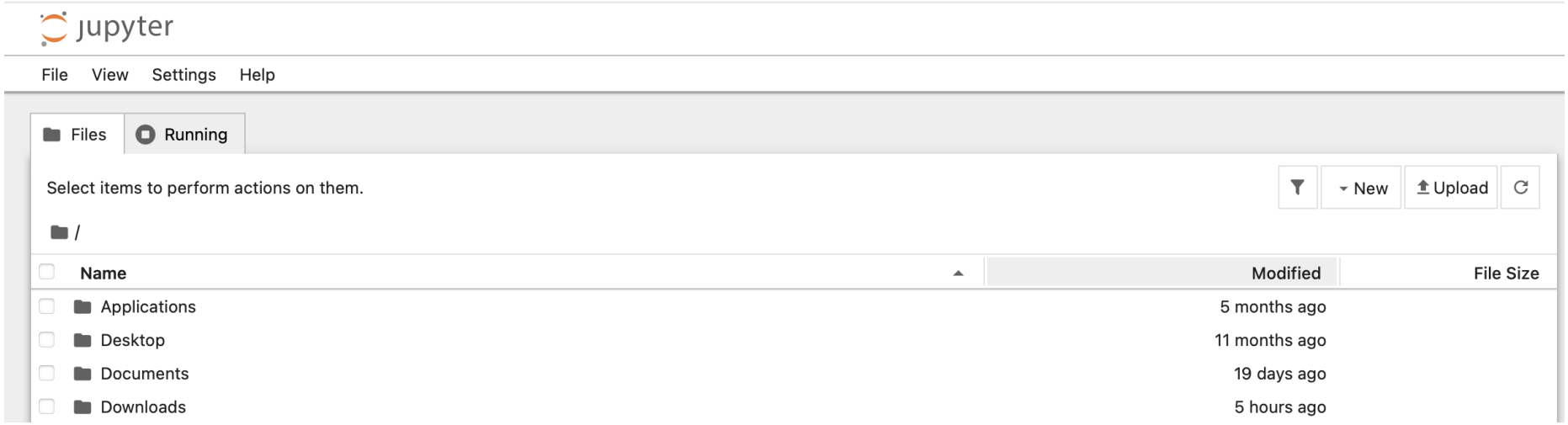

EXPLORER
NO FOLDER OPENED
You have not yet opened a folder.
Open Folder
Show All Commands ⇧ ⌘ P
Open File or Folder ⌘ O
Open Recent ^ R
New Untitled Text File ⌘ N
Open Chat ^ ⌘ I
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
bash
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
(base) Usmans-MacBook-Pro:~ uq$ Press ⌘ I to ask to do something. Start typing to dismiss.


EXTENSIONS: MARKETPLACE
python
Python Debugger
Python Debugger extension using...
Microsoft
Python
Python language support with ext...
Microsoft
Pylance
A performant, feature-rich langua...
Microsoft
Python Environments
Provides a unified python environ...
Microsoft
Python Indent
14.8M
4
Correct Python indentation
Kevin Rose
Install
Python for VSCode
6.1M
2
Python language extension for vs...
omas Haakon Townsend
Install
Python Extension ...
11.8M
4.5
Popular Visual Studio Code exten...
Don Jayamanne
Install
Python Environme...
11.9M
4
View and manage Python environ...
Don Jayamanne
Install
autoDocstring - Py...
13.9M
5
Generates python docstrings aut...
Nils Werner
Install
Python Preview
2.4M
4.5
Provide Preview for Python Execu...
dongli
Install
Extension: Python
Python
Microsoft
microsoft.com
179,326,599
(618)
Python language support with extension access points for IntelliSense (Pylance), Debugging (Python Debugger...
Disable
Uninstall
Switch to Pre-Release Version
Auto Update
DETAILS
FEATURES
CHANGELOG
EXTENSION PACK
Python extension for Visual Studio Code
A Visual Studio Code extension with rich support for the Python language (for all actively supported Python versions), providing access points for extensions to seamlessly integrate and offer support for IntelliSense (Pylance), debugging (Python Debugger), formatting, linting, code navigation, refactoring, variable explorer, test explorer, and more!
Support for vscode.dev
The Python extension does offer some support when running on vscode.dev (which includes github.dev). This includes partial IntelliSense for open files in the editor.
Installed extensions
The Python extension will automatically install the following extensions by default to provide the best Python development experience in VS Code:
Pylance - to provide performant Python language support
Python Debugger - to provide a seamless debug experience with debugpy
These extensions are optional dependencies, meaning the Python extension will remain fully functional if they fail to be installed. Any or all of these extensions can be disabled or uninstalled at the expense of some features. Extensions installed through the marketplace are subject to the Marketplace Terms of Use.
Extensibility
Installation
Identifier
ms-python.python
Version
2025.12.0
Last Updated
2025-08-15, 15:15:55
Size
28.74MB
Cache
2.20KB
Marketplace
Published
2016-01-19, 20:03:11
Last Released
2025-08-15, 15:54:52
Categories
Programming Languages
Debuggers
Other
Data Science
Machine Learning
Resources

Running **Jupyter Notebook**

- **pip install jupyter**

- **jupyter notebook**



A First Python Program

- The input() function pauses program execution and waits for user input from the keyboard.

```
# Basic interactive program
print('Hello, world')
user_name = input('Enter your name: ')
print('Hello', user_name)

# Storing multiple inputs
first_name = input('Enter first name: ')
last_name = input('Enter last name: ')
print('Full name:', first_name, last_name)

# Input with prompt formatting
age = input(f'Hello {first_name}, how old are you? ')
print(f'{first_name} is {age} years old')
```

```
Hello, world
Enter your name:  Ali Raza
Hello Ali Raza
Enter first name:  Ali
Enter last name:  Raza
Full name: Ali Raza
Hello Ali, how old are you?  25
Ali is 25 years old
```

Variables

- Variables are used to store data values in memory locations that can be referenced by name. Python uses dynamic typing, meaning variables can hold values of any type and can change type.

```
# Variable examples showing dynamic typing
my_var = 'John'  # String
print(my_var, type(my_var))

my_var = 42      # Integer
print(my_var, type(my_var))

my_var = True    # Boolean
print(my_var, type(my_var))

# Multiple assignments
x, y, z = 10, 20, 30
print(x, y, z)

# Swapping values
a, b = 1, 2
a, b = b, a
print('a:', a, 'b:', b)
```

```
John <class 'str'>
42 <class 'int'>
True <class 'bool'>
10 20 30
a: 2 b: 1
```

Naming Conventions

- Python has specific naming conventions that help make code more readable and maintainable. Variables should be lowercase with words separated by underscores (snake_case).

```
# Good naming examples
user_name = 'Alice'
account_balance = 100.50
is_active = True
max_connections = 10

# Problematic naming examples
UserName = 'Bob'       # PascalCase - used for classes
accountBalance = 200   # camelCase - not Python convention
IS_ACTIVE = False      # ALL_CAPS - used for constants
x = 30                 # Too vague

# Legal but discouraged
_private_var = 'hidden'  # Leading underscore indicates "private"
__mangled = 'special'    # Double underscore triggers name mangling
print(user_name, account_balance)
```

```
Alice 100.5
```

Assignment Operator

- The assignment operator (=) stores values in variables. Python supports multiple assignment and compound operators that combine arithmetic with assignment.

```
# Basic assignment
counter = 0
name = 'Charlie'

# Multiple assignment
x, y, z = 1, 2, 3
print(x, y, z)

# Compound assignment operators
total = 10
total += 5  # Equivalent to total = total + 5
print(total)

count = 3
count *= 4  # count = count * 4
print(count)

# Chained assignment
a = b = c = 100
print(a, b, c)
```

```
1 2 3
15
12
100 100 100
```

Python Statements

- A statement is a unit of code that performs some action. Python has different types of statements including expression statements, assignment statements, and control flow statements.

```
# Simple statements
print('This is a statement')
result = 4 + 5  # Assignment statement

# Multi-line statements
total = (1 + 2 + 3 +
         4 + 5 + 6)
print(total)

# Multiple statements on one line (discouraged)
x = 1; y = 2; print(x + y)

# Expression vs statement
3 + 4      # Expression (produces value)
x = 3 + 4  # Statement (performs action)
```

```
This is a statement
21
3
```

Python Strings

Strings

- Strings are immutable sequences of Unicode characters used to store and manipulate text in Python. They can be created using single quotes, double quotes, or triple quotes for multi-line strings.

```
# String examples
str1 = 'Hello World'
str2 = "Python Programming"
str3 = '''Multi-line
string example'''
empty_str = ''

print(str1)
print(str2)
print(str3)
print("Empty string:", empty_str)
```

```
Hello World
Python Programming
Multi-line
string example
Empty string:
```

Representing Strings

- Python offers flexible ways to represent strings using different quote styles.

```
# Different string representations
single_quote = 'It\'s a single quote string'
double_quote = "She said \"hello\" to everyone"
triple_quote = """This is a
multi-line
string"""

print(single_quote)
print(double_quote)
print(triple_quote)
```

```
It's a single quote string
She said "hello" to everyone
This is a
multi-line
string
```

Data type of String

- In Python, all strings are of type 'str', which can be verified using the type() function.

```
# Checking string type
my_variable = 'Bob'
print(type(my_variable))

# Dynamic typing example
my_variable = 42
print("After reassignment:", type(my_variable))
```

```
<class 'str'>
After reassignment: <class 'int'>
```

String Concatenation

- String concatenation combines multiple strings into one using the + operator.

```
# String concatenation examples
string_1 = 'Good'
string_2 = " day"
string_3 = string_1 + string_2
print(string_3)
print('Hello ' + 'World')

# Concatenation with conversion
age = 25
print('Age: ' + str(age))
```

```
Good day
Hello World
Age: 25
```

Length of a String

- The len() function returns the number of characters in a string, which is useful for validation and processing. This count includes all characters, including spaces and special characters.

```
# String length examples
my_string = 'Hello World'
print(len(my_string))
print(len(''))  # Empty string
print(len('Python'))
print(len('With spaces'))
print(len('\n\t'))  # Special characters
```

```
11
0
6
11
2
```

Accessing a Character

- Individual characters in a string can be accessed using square bracket notation with zero-based indexing. Python also supports negative indexing (-1 for last character).

```
# Accessing characters in a string
my_string = 'Hello World'
print(my_string[0])  # First character
print(my_string[6])  # 7th character (space)
print(my_string[-1])  # Last character (negative indexing)
# print(my_string[20])  # This would raise an IndexError
```

```
H
W
d
```

Accessing a Subset of Characters

- Substrings can be extracted using slice notation [start:end:step].

```
# Substring examples
my_string = 'Hello World'
print(my_string[1:5])   # From index 1 to 4
print(my_string[:5])    # From start to index 4
print(my_string[6:])    # From index 6 to end
print(my_string[::2])   # Every second character
print(my_string[::-1])  # Reversed string
```

```
ello
Hello
World
HloWrd
dlroW olleH
```

Repeating Strings

- The * operator can be used to repeat strings a specified number of times.

```
# String repetition examples
print('*' * 10)
print('Hi' * 5)
print('=' * 20)
print('+-' * 10)
print('Border: ' + '#' * 30)
```

```
**********
HiHiHiHiHi
====================
+-+-+-+-+-+-+-+-+-+-
Border: ##############################
```

Splitting Strings

- The split() method divides a string into a list of substrings based on a delimiter. Common delimiters include spaces, commas, or other characters.

```
# String splitting examples
title = 'The Good, The Bad, and the Ugly'
print(title.split(' '))  # Split by space
print(title.split(','))  # Split by comma

# Splitting with maxsplit parameter
print(title.split(' ', 2))  # Split into 3 parts
print('  multiple   spaces  '.split())  # Default splits on whitespace
```

```
['The', 'Good,', 'The', 'Bad,', 'and', 'the', 'Ugly']
['The Good', ' The Bad', ' and the Ugly']
['The', 'Good,', 'The Bad, and the Ugly']
['multiple', 'spaces']
```

Counting Strings

- The count() method returns the number of non-overlapping occurrences of a substring.

```
# String counting examples
my_string = 'Count, the number    of spaces'
print(my_string.count(' '))  # Count spaces
print(my_string.count('e'))  # Count 'e's
print('mississippi'.count('ss'))  # Count substring
print('abababab'.count('aba'))  # Non-overlapping counts
```

```
7
3
2
2
```

Replacing Strings

- The replace() method returns a new string with all occurrences of a substring replaced. It can limit replacements with the count parameter.

```
# String replacement examples
welcome_message = 'Hello World!'
print(welcome_message.replace("Hello", "Goodbye"))

text = 'one one one'
print(text.replace('one', 'three', 2))  # Replace first 2 occurrences
print('case MATTERS'.replace('matters', 'important'))  # No replacement
```

```
Goodbye World!
three three one
case MATTERS
```

Finding Sub Strings

- The find() method locates the first occurrence of a substring, returning its index or -1 if not found.

```
# Finding substrings
print('Edward Alun Rawlings'.find('Alun'))  # Returns 7
print('Edward John Rawlings'.find('Alun'))  # Returns -1

# Finding with start and end parameters
print('hello hello'.find('ell', 3))  # Start searching from index 3
print('hello'.find('l'))  # First 'l' at index 2
print('hello'.rfind('l'))  # Last 'l' at index 3
```

```
7
-1
7
2
3
```

Converting Other Types into Strings

- The str() function converts other data types to their string representation. This is essential for concatenation with strings and display purposes.

```
# Type conversion to string
age = 21
pi = 3.14159
msg = 'Age: ' + str(age) + ', Pi: ' + str(pi)
print(msg)

# Using f-strings (Python 3.6+)
print(f'Age: {age}, Pi: {pi:.2f}')

# Converting containers
print(str([1, 2, 3]))
print(str({'name': 'Alice', 'age': 25}))
```

```
Age: 21, Pi: 3.14159
Age: 21, Pi: 3.14
[1, 2, 3]
{'name': 'Alice', 'age': 25}
```

Comparing Strings

- trings can be compared using == (equality) and != (inequality) operators. Comparisons are case-sensitive and compare Unicode code points.

```
# String comparison examples
print('James' == 'James')  # True
print('James' == 'John')   # False
print('James' != 'John')   # True
print('James' == 'james')  # False (case-sensitive)

# Case-insensitive comparison
print('James'.lower() == 'james'.lower())  # True

# Lexicographical ordering
print('apple' < 'banana')  # True
print('Apple' < 'apple')   # True (uppercase comes first)
```

```
True
False
True
False
True
True
True
```

Other String Operations

- Python offers numerous string methods for various purposes.

```
# Various string operations
some_string = 'Hello World'
print("Testing a String")
print("-" * 20)
print("some_string.startswith('H')", some_string.startswith('H'))
print("some_string.endswith('d')", some_string.endswith('d'))
print("some_string.istitle()", some_string.istitle())
print("some_string.isupper()", some_string.isupper())
print("some_string.isalpha()", some_string.isalpha())

print("String conversions")
print("-" * 20)
print("some_string.upper()", some_string.upper())
print("some_string.lower()", some_string.lower())
print("some_string.title()", some_string.title())
print("String leading, trailing spaces", "   xyz   ".strip())
```

```
Testing a String
--------------------
some_string.startswith('H') True
some_string.endswith('d') True
some_string.istitle() True
some_string.isupper() False
some_string.isalpha() False
String conversions
--------------------
some_string.upper() HELLO WORLD
some_string.lower() hello world
some_string.title() Hello World
String leading, trailing spaces xyz
```

Other String Operations

- Python offers numerous string methods for various purposes.

```python
# String method hints
s = 'Hello'
print(s.isupper)    # Shows method reference
print(s.isupper())  # Actually calls the method

# Case sensitivity example
print('Hello'.startswith('h'))  # False
print('Hello'.lower().startswith('h'))  # True

# String module constants
import string
print(string.ascii_letters)  # All ASCII letters
print(string.digits)         # All digits
```

```
<built-in method isupper of str object at 0x0000018793752700>
False
False
True
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
0123456789
```

String Formatting

- String formatting allows dynamic value insertion into strings using the format() method.

```
# String formatting examples
# Basic formatting
print("{} is {} years old".format("Alice", 25))

# Positional indexing
print("Hello {1} {0}".format("Smith", "Carol"))

# Named parameters
print("{artist} sang {song}".format(artist="Paloma Faith", song="Guilty"))

# Formatting options
print("|{:<20}|".format("left"))     # Left aligned
print("|{:>20}|".format("right"))    # Right aligned
print("|{:^20}|".format("center"))   # Centered
print("{:,}".format(1234567890))     # Number formatting
print("{:.2f}".format(3.14159))     # Floating point precision
```

```
Alice is 25 years old
Hello Carol Smith
Paloma Faith sang Guilty
|left                |
|               right|
|       center       |
1,234,567,890
3.14
```

String Templates

- String templates provide simpler string substitution using $variables. They are created using the Template class from the string module.

```
# String template examples
from string import Template

# Create template
template = Template('$artist sang $song in $year')

# Substitute values
print(template.substitute(artist='Freddie Mercury',
                          song='The Great Pretender',
                          year=1987))

# Safe substitution (won't error on missing values)
print(template.safe_substitute(artist='David Bowie', song='Rebel Rebel'))

# Reusing template
print(template.substitute(artist='Ed Sheeran', song='Galway Girl', year=2017))
```

```
Freddie Mercury sang The Great Pretender in 1987
David Bowie sang Rebel Rebel in $year
Ed Sheeran sang Galway Girl in 2017
```

Numbers

Integer Numbers

- Integers are whole numbers (positive or negative) without a decimal point.

```
x = 25
y = -10
print("Sum:", x + y)
print("Product:", x * y)
print("Modulus:", x % 4)
```

```
Sum: 15
Product: -250
Modulus: 1
```

Floating Point Numbers

- Floating point numbers (floats) represent real numbers with decimal points.

```
pi = 3.14159
radius = 2.5
area = pi * (radius ** 2)
print("Circle area:", area)
```

```
Circle area: 19.6349375
```

Converting Numbers

- Python supports converting between integers and floats using int() and float() functions. You can also convert numeric types to strings using str().

```
a = 8.7
print("int(a):", int(a))
b = 5
print("float(b):", float(b))
print("String conversion:", str(b))
```

```
int(a): 8
float(b): 5.0
String conversion: 5
```

Division Behavior

- Python has two division operators: / for true division (returns a float) and // for floor division (truncates toward negative infinity).

```
print("7 / 3 =", 7 / 3)
print("7 // 3 =", 7 // 3)
print("-7 // 3 =", -7 // 3)
```

```
7 / 3 = 2.3333333333333335
7 // 3 = 2
-7 // 3 = -3
```

Complex Numbers

- Python supports complex numbers using j to denote the imaginary part. A complex number has real and imaginary components accessible via .real and .imag.

```
z1 = 2 + 3j
z2 = 1 - 4j
print("Sum:", z1 + z2)
print("Product:", z1 * z2)
print("Real part:", z1.real)
print("Imaginary part:", z1.imag)
```

```
Sum: (3-1j)
Product: (14-5j)
Real part: 2.0
Imaginary part: 3.0
```

The bool Type

- *bool* is a subclass of int in Python with two possible values: True and False.

```
print("True as int:", int(True))
print("False as int:", int(False))
a = 5
b = 10
print("a < b:", a < b)
```

```
True as int: 1
False as int: 0
a < b: True
```

Boolean Evaluation of Non-Boolean Values

- Python evaluates non-Boolean values in a Boolean context using their 'truthiness'. Common false values include 0, 0.0, None, empty strings, lists, and dictionaries. Everything else evaluates as true in a Boolean context.

```
print("bool(0):", bool(0))
print("bool(42):", bool(42))
print("bool(''):", bool(''))
print("bool([1, 2]):", bool([1, 2]))
```

```
bool(0): False
bool(42): True
bool(''): False
bool([1, 2]): True
```

The None Value

- *None* is a special Python object used to indicate the absence of a value or a null result. It's commonly used as a default value in function arguments or when a function returns nothing. It evaluates to False in logical tests.

```
result = None
if result is None:
    print("Result has no value")
else:
    print("Result is:", result)
```

```
Result has no value
```

Flow Controls

The if Statement

- The if statement is used to execute a block of code only if a specified condition evaluates to True. This allows your program to make decisions and take actions based on data or input.

```
x = 10
if x > 5:
    print("x is greater than 5")
```

```
x is greater than 5
```

The else Clause

- The else clause provides an alternative block of code to execute when the condition in the if statement is False. This ensures that some action is taken in either case.

```
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")
```

```
x is not greater than 5
```

The elif Clause

- The elif (short for else-if) clause allows checking multiple conditions sequentially. Each elif is tested only if all preceding conditions were False.

```
score = 85
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

```
Grade: B
```

Nested if Statements

- Nested if statements are conditionals placed inside other conditionals.

```
x = 10
if x > 0:
    print("x is positive")
    if x % 2 == 0:
        print("x is even")
```

```
x is positive
x is even
```

Logical Operators in Conditions

- Python supports logical operators like and, or, and not to combine multiple conditions.

```
age = 25
income = 40000
if age > 18 and income > 30000:
    print("Eligible for loan")
else:
    print("Not eligible")
```

```
Eligible for loan
```

Looping/ Iterations

The while Statement

- The while loop repeats a block of code as long as a condition is true. It's typically used when the number of iterations is unknown ahead of time.

```
n = 0
while n < 5:
    print(n)
    n += 1
```

```
0
1
2
3
4
```

Using while with User Input

- You can use a while loop with input to create interactive programs that run until the user decides to stop.

```
command = ""
while command != "quit":
    command = input("Enter command (quit to exit): ")
    print("You typed:", command)
```

```
Enter command (quit to exit):  hello
You typed: hello
Enter command (quit to exit):  quit
You typed: quit
```

The for Statement

- The for loop is used to iterate over a sequence such as a list, string, or range. Unlike the while loop, the number of iterations is usually known. It makes looping over collections simple and readable.

```
colors = ["red", "green", "blue"]
for color in colors:
    print("Color:", color)
```

```
Color: red
Color: green
Color: blue
```

The range() Function

- The range() function is often used with for loops to generate numeric sequences. It can take a start, stop, and optional step value. It's especially useful for running a loop a fixed number of times.

```
for i in range(3):
    print("Iteration:", i)

for i in range(5, 0, -1):
    print("Countdown:", i)
```

```
Iteration: 0
Iteration: 1
Iteration: 2
Countdown: 5
Countdown: 4
Countdown: 3
Countdown: 2
Countdown: 1
```

Looping Through Strings

- Strings in Python are sequences of characters, so you can iterate through them using a for loop.

```
word = "hello"
for letter in word:
    print(letter.upper())
```

```
H
E
L
L
O
```

Nested Loops

- A nested loop is a loop inside another loop. Each time the outer loop runs once, the inner loop runs fully. Nested loops are useful for working with multi-dimensional data like matrices or generating grid patterns, but they can increase time complexity.

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"i={i}, j={j}")
```

```
i=1, j=1
i=1, j=2
i=1, j=3
i=2, j=1
i=2, j=2
i=2, j=3
i=3, j=1
i=3, j=2
i=3, j=3
```

Functions

Defining Functions

- Defining a function in Python begins with the def keyword, followed by the function name and parentheses.
- The body of the function contains the logic that runs when the function is called.

```
def say_hello():
    print("Hello there!")

say_hello()
say_hello()
```

```
Hello there!
Hello there!
```

Returning Values from Functions

- Functions can return values using the return keyword. This allows the function to output a result that can be stored or used in expressions.

```
def square(num):
    return num * num

result = square(5)
print("Square:", result)
```

```
Square: 25
```

Docstring

- A docstring is a special string literal that appears as the first statement in a function. It is used to document what the function does, its parameters, and return values.
- Python tools like help() use the docstring to display helpful information.

```
def add(a, b):
    """Returns the sum of two numbers."""
    return a + b

print(help(add))
```

```
Help on function add in module __main__:

add(a, b)
    Returns the sum of two numbers.

None
```

Function Parameters

- Functions often need input values to work with. Parameters are defined inside the parentheses in the function definition, and arguments are passed when calling the function.

```
def greet_user(name):
    print("Hello,", name)

greet_user("Alice")
```

```
Hello, Alice
```

```
def multiply(a, b):
    return a * b

print(multiply(3, 4))
```

```
12
```

Default Parameter Values

- Default parameters allow you to call a function without explicitly passing all arguments. If a value is not provided, the default is used.

```
def greet(name="Guest"):
    print("Welcome,", name)

greet("John")
greet()
```

```
Welcome, John
Welcome, Guest
```

Named Arguments

- Named arguments (also called keyword arguments) allow you to specify which parameter corresponds to which argument by name, not just by position. This improves readability and lets you skip or reorder arguments as needed.

```
def format_name(first, last):
    return f"{last}, {first}"

print(format_name(last="Doe", first="Jane"))
```

```
Doe, Jane
```

Arbitrary Arguments

- Sometimes you may want a function to accept an unknown number of arguments. Python allows this using *args for positional arguments and **kwargs for keyword arguments.

```
def add_all(*numbers):
    return sum(numbers)

print(add_all(1, 2, 3, 4))
```

```
10
```

Positional and Keyword Arguments

- Python supports both positional and keyword arguments in function calls. Positional arguments are matched by order, while keyword arguments are matched by name.

```python
def info(name, age, city="Unknown"):
    print(f"Name: {name}, Age: {age}, City: {city}")

info("Alice", 30)
info("Bob", 25, city="Paris")
```

```
Name: Alice, Age: 30, City: Unknown
Name: Bob, Age: 25, City: Paris
```

Anonymous Functions

- Anonymous functions in Python are defined using the lambda keyword. They are typically short, single-expression functions not bound to a name.

```
square = lambda x: x * x
print(square(6))

names = ["Alice", "Bob", "Charlie"]
names.sort(key=lambda name: len(name))
print(names)
```

```
36
['Bob', 'Alice', 'Charlie']
```

Classes

Class Definitions

- In Python, classes are defined using the class keyword followed by the class name and a colon. A class is a blueprint for creating objects, grouping data (attributes) and functions (methods) together.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
person1 = Person("Alice", 30)
person2 = Person("Bob", 25)
print(person1.name, person1.age)
print(person2.name, person2.age)
```

```
Alice 30
Bob 25
```

- In Python, assignment of one object to another does not create a new copy. Instead, both variables point to the same object in memory. This means that changes made through one reference are visible through the other.

```
person3 = person1  # Not a new object
person3.age = 40
print(person1.age)  # Will also show 40
```

```
40
```

Printing Out Objects

- When you try to print an object in Python, it displays a default representation which is not very informative.
- You can override this behavior by defining the __str__() method in your class.
- This method should return a human-readable string describing the object.
- It is automatically called when you use print() on the object.

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def __str__(self):
        return f"Person(name={self.name}, age={self.age})"

person4 = Person("Charlie", 40)
print(person4)
```

```
Person(name=Charlie, age=40)
```

Defining Instance Methods

- Instance methods are functions defined within a class that operate on instances of that class. They always take self as their first parameter to access the object's attributes and other methods.
- You can define as many instance methods as needed to model your object's behavior.

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        return f"Hi, I'm {self.name} and I'm {self.age} years old."

print(Person("Eva", 22).introduce())
```

```
Hi, I'm Eva and I'm 22 years old.
```

The del Keyword

- The del keyword is used to delete objects or their attributes. When you use del object, Python removes the reference to the object.

```
person = Person("Frank", 60)
del person  # Deletes the object
# the following line will create error if uncommented
print(person.age)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[207], line 4
      2 del person  # Deletes the object
      3 # the following line will create error if uncommented
----> 4 print(person.age)

NameError: name 'person' is not defined
```

Intrinsic Attributes

- All Python objects have built-in attributes that provide metadata about the object. These include __dict__, __class__, __module__, and __doc__.
- They can be accessed using dot notation.

```
p = Person("Grace", 50)
print(p.__dict__)
print(p.__class__)
print(p.__doc__)
```

```
{'name': 'Grace', 'age': 50}
<class '__main__.Person'>
None
```

Creating a Derived Class

- To create a derived class in Python, define a new class and specify the base class in parentheses after the class name.
- The derived class inherits all the methods and properties of the base class.

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

class Student(Person):
    def __init__(self, name, age, student_id):
        Person.__init__(self, name, age)
        self.student_id = student_id

s = Student("Alice", 20, "S123")
s.display()
print("Student ID:", s.student_id)
```

```
Name: Alice, Age: 20
Student ID: S123
```

Overriding Methods

- In a derived class, you can override methods from the base class by defining a method with the same name. This allows the derived class to change or extend the behavior of the base class.

```
class Person:
    def display(self):
        print("This is a person.")

class Student(Person):
    def display(self):  # Overrides the method
        print("This is a student.")

p = Person()
p.display()
s = Student()
s.display()
```

```
This is a person.
This is a student.
```

Tuples, Lists, Sets, and Dictionaries

Tuples

- Tuples are immutable sequences in Python.
- They are similar to lists but cannot be changed once created. Tuples are useful when you want to ensure data integrity and avoid accidental modifications.
- They are defined using parentheses () or the tuple() constructor.

```
empty = ()
one = (10,)
several = (10, 20, 30)
print(empty, one, several)
```

```
() (10,) (10, 20, 30)
```

```
s = "Hello"
t = tuple(s)
print(t)
```

```
('H', 'e', 'l', 'l', 'o')
```

Tuples

- Accessing Elements of a Tuple.

```
t = (10, 20, 30, 40)
print(t[0], t[-1])
```

```
10 40
```

- Creating New Tuples from Existing Tuples.

```
t = (1, 2, 3)
new_t = t + (4, 5)
print(new_t)
```

```
(1, 2, 3, 4, 5)
```

- Tuples Can Hold Different Types.

```
mixed = (1, 3.14, "hello", (10, 20))
print(mixed)
```

```
(1, 3.14, 'hello', (10, 20))
```

Tuples

- You can iterate over tuples using a for loop

```
t = (10, 20, 30)
for item in t:
    print(item)
```

```
10
20
30
```

- Different Tuple Operations

```
nums = (5, 10, 15)
print(len(nums), sum(nums), min(nums), max(nums))
```

```
3 30 5 15
```

- Checking if an Element Exists

```
t = (1, 2, 3, 4)
print(3 in t, 5 in t)
```

```
True False
```

Tuples

- Tuples can contain other tuples as elements.

```
nested = ((1, 2), (3, 4))
print(nested[0][1])
```

```
2
```

- Since tuples are immutable, you cannot change, append, or remove elements after creation.

```
t = (1, 2, 3)
 t[0] = 10  # Uncommenting this line will raise a TypeError
```

```
  Cell In[235], line 2
    t[0] = 10  # Uncommenting this line will raise a TypeError
    ^
IndentationError: unexpected indent
```

Lists

- Lists are one of the most flexible and widely used data structures in Python. They are mutable sequences, meaning you can modify their contents after creation.
- To create a list, enclose items in square brackets, separated by commas. Lists can contain any type of object, including other lists or tuples.
- Lists are ordered and indexed starting from 0.

```
empty = []
numbers = [1, 2, 3]
mixed = [10, "hello", 3.14]
print(numbers, mixed)
```

```
[1, 2, 3] [10, 'hello', 3.14]
```

```
s = "Hello"
l = list(s)
print(l)
```

```
['H', 'e', 'l', 'l', 'o']
```

Lists

- List elements are accessed using index notation. Negative indices count from the end of the list.

```
lst = [10, 20, 30, 40]
print(lst[0], lst[-1])
```

```
10 40
```

- You can add elements to the end of a list using the append() method.

```
lst = [1, 2, 3]
lst.append(4)
print(lst)
```

```
[1, 2, 3, 4]
```

- The insert() method lets you add an element at a specific index.

```
lst = [1, 2, 4]
lst.insert(2, 3)
print(lst)
```

```
[1, 2, 3, 4]
```

Lists

- You can concatenate two or more lists using the + operator.

```
a = [1, 2]
b = [3, 4]
c = a + b
print(c)
```

```
[1, 2, 3, 4]
```

- Use remove() to delete the first occurrence of a value from a list.

```
lst = [10, 20, 30]
lst.remove(20)
print(lst)
```

```
[10, 30]
```

- The pop() method removes and returns the last item in the list by default.

```
lst = [1, 2, 3, 4]
x = lst.pop()
print(x, lst)
```

```
4 [1, 2, 3]
```

Lists

- You can delete elements from a list using the del statement and an index.

```
lst = [0, 1, 2, 3, 4]
del lst[2]
print(lst)
```

```
[0, 1, 3, 4]
```

- Python lists come with several useful methods: append(), insert(), remove(), pop(), sort(), reverse(), and more.

```
lst = [3, 1, 2]
lst.sort()
lst.reverse()
print(lst)
```

```
[3, 2, 1]
```

Sets

- A set is an unordered collection data type that is iterable, mutable, and has no duplicate elements.
- Sets in Python are defined by enclosing elements in curly braces {}.

```
s = {1, 2, 3, 4, 4, 2}
print(s)  # {1, 2, 3, 4} - duplicates removed
```

```
{1, 2, 3, 4}
```

- You can also use set(...) function to create sets.

```
a = set([1, 2, 3])
b = set("hello")
empty = set()
print(a)
print(b)
print(empty)
```

```
{1, 2, 3}
{'l', 'h', 'o', 'e'}
set()
```

Sets

- Because sets are unordered, items cannot be accessed by index or slice. You can check for membership using the *in* keyword.

```
s = {1, 2, 3}
for item in s:
    print(item)
print(2 in s)
```

```
1
2
3
True
```

- Sets are mutable, so you can add or remove elements.

```
s = {1, 2, 3}
s.add(4)
s.update([5, 6])
s.remove(2)
s.discard(10)
print(s)
```

```
{1, 3, 4, 5, 6}
```

Sets

- Sets support standard mathematical operations like union, intersection, difference, and symmetric difference.

```
a = {1, 2, 3}
b = {3, 4, 5}
print("Union:", a | b)
print("Intersection:", a & b)
print("Difference:", a - b)
print("Symmetric Difference:", a ^ b)
```

```
Union: {1, 2, 3, 4, 5}
Intersection: {3}
Difference: {1, 2}
Symmetric Difference: {1, 2, 4, 5}
```

- Sets come with a rich set of built-in methods.

```
s = {1, 2, 3}
s.pop()
copy = s.copy()
s.clear()
print(copy)
print(s)
```

```
{2, 3}
set()
```

Dictionaries

- A dictionary is a mutable, unordered collection of key-value pairs. Keys must be unique and of an immutable type, such as strings, numbers, or tuples.
- You can create dictionaries using curly braces {} or the dict() constructor.

```
d1 = {"name": "Alice", "age": 25}
d2 = dict(city="New York", country="USA")
d3 = dict([("x", 1), ("y", 2)])
print(d1)
print(d2)
print(d3)
```

```
{'name': 'Alice', 'age': 25}
{'city': 'New York', 'country': 'USA'}
{'x': 1, 'y': 2}
```

Dictionaries

- To access a value in a dictionary, use the key inside square brackets.

```
person = {"name": "Bob", "age": 30}
print(person["name"])
```

```
Bob
```

- You can modify an existing value in a dictionary by assigning a new value to its key.

```
person["age"] = 35
print(person)
```

```
{'name': 'Bob', 'age': 35}
```

- Adding a new item is as simple as assigning a value to a new key.

```
person["gender"] = "Male"
print(person)
```

```
{'name': 'Bob', 'age': 35, 'gender': 'Male'}
```

Dictionaries

- Use *del* to remove a key-value pair from the dictionary.

```
del person["gender"]
print(person)
```

```
{'name': 'Bob', 'age': 35}
```

- Use the in keyword to check if a key exists in the dictionary.

```
print("name" in person)
print("salary" in person)
```

```
True
False
```

- The get() method returns the value for a given key if it exists, otherwise it returns None or a default value if specified.

```
print(person.get("name"))
print(person.get("salary", "Not available"))
```

```
Bob
Not available
```

Dictionaries

- keys() returns a view object that displays all the keys in the dictionary.

```
print(person.keys())
```

```
dict_keys(['name', 'age'])
```

- items() returns a view object with all the key-value pairs in the dictionary as tuples.

```
for key, value in person.items():
    print(f"{key}: {value}")
```

```
name: Bob
age: 35
```

Collection Modules

The array Module

- The array module offers an array-like structure that is more memory-efficient than lists. Arrays must contain elements of the same type, and you must specify a type code (such as 'i' for integers or 'f' for floats).

```
import array
arr = array.array('i', [1, 2, 3, 4])
for num in arr:
    print(num)
```

```
1
2
3
4
```

File Handling

File Handling: Writing to a file

- To write to a file, open it in 'w' (write) or 'a' (append) mode.
  Writing with 'w' will overwrite the file if it exists, while 'a' adds content to the end.
- The write() method takes a string and writes it to the file.

```
with open('testfile.txt', 'w') as file:
    file.write('Hello, world!')
```

File Handling: Reading a file

- Python provides built-in functions to handle text and binary files using simple and efficient methods.
- To read a file in Python, we use the open() function with mode 'r' (read mode). Using a with statement ensures that the file is properly closed after it's used.

```
with open('testfile.txt', 'r') as file:
    content = file.read()
    print(content)
```

```
Hello, world!
```

- Reading a file line by line is efficient for large files. You can loop through the file object directly, which returns one line at a time. Using strip() removes extra newline characters from each line.

```
with open('testfile.txt', 'r') as file:
    for line in file:
        print(line.strip())
```

```
Hello, world!
New content
```

File Modes

- Python’s open() function accepts various modes.
- Use 'r' to read, 'w' to write (truncate), 'a' to append, 'b' for binary files, and '+' to update (read/write).
- These can be combined like 'r+' to read and write.

```
with open('testfile.txt', 'r+') as file:
    content = file.read()
    file.write('\nNew content')
```

Exception Handling with Files

- File operations can raise exceptions, such as FileNotFoundError when a file doesn’t exist. t’s good practice to use try-except blocks to handle such errors gracefully.

```
try:
    with open('example.txt', 'r') as file:
        print(file.read())
except FileNotFoundError:
    print('File not found.')
```

```
File not found.
```

Thank You